1 Full VRAM Sparse Neural Network: Overcoming the Memory Wall

Manuscript (not final). Rev. 1.1, October 25, 2026

Abstract
Tackling sparse neural networks (SNN) is a highly challenging task. While competing with dense neural networks, which can be implemented very efficiently on state of the art GPU hardware, SNN typically suffer from reduced memory usage and / or inefficient GPU core usage. I believe to have found a new approach (Dendritic Gather-WMMA, DG-WMMA) which performs well reaching up to 64 TFlops out of the theoretical 103 TFlops. Even below peak performance my kernel keeps pace for large systems up to 10–40 million neurons, leaving the GPU cache while still sustaining up to 50 TFlops at 10 million neurons ($M = 512$) and scaling up to 40 million neurons (99.997% sparsity at $M = 128$). I will present the architecture of the kernel and show benchmarks how I reached more than 50 TFlops for sparsities ranging from 0% to 99.98%, alongside an outlook on real-world network execution and a preliminary CUDA port reaching over 1 PFlops (> 500 TSyn/s) on NVIDIA Blackwell B200 across up to 80–320 million neurons.

1.1 1. Introduction

I worked in the field of microscopy specialized in nervous systems with an emphasis of image processing using GPUs. With the advent of agentic coding I found the time and opportunity, next to my day job, to try out my approach of implementing neural networks inspired by nature. This is the first publication (current status: manuscript) focusing on the efficient Kernel implementation on RDNA4 hardware.

The scaling laws of neural networks have dictated that larger parameter counts directly correlate with enhanced generalization and emergent capabilities (Kaplan et al., 2020; Hoffmann et al., 2022). Consequently, the size of Large Language Models (LLMs) has grown significantly, causing inference workloads to shift from compute-bound to memory-bandwidth-bound regimes. This phenomenon, known as the Memory Wall, dictates that GPUs spend the vast majority of their execution cycles waiting for data to be transferred from external High Bandwidth Memory (HBM) or GDDR6, rather than performing arithmetic operations (Patterson et al., 1997; Jouppi et al., 2017).

To combat the Memory Wall, the machine learning community has extensively researched network pruning and unstructured sparsity (Han et al., 2015). The *Lottery Ticket Hypothesis* (Frankle & Carbin, 2018) and subsequent works have demonstrated that extreme unstructured sparsity (often between 80% and 90%) can be achieved without sacrificing model fidelity. Theoretically, eliminating 90% of the weights should yield a proportionate reduction in memory traffic and latency.

However, the field faces a fundamental "chicken-and-egg" problem: there is little economic incentive for researchers to develop highly sparse algorithms because commodity GPUs natively penalize unstructured sparsity. Deep learning frameworks like PyTorch and libraries like cuSPARSE rely on classical sparse storage formats. Because these formats map non-uniformly to the underlying Single Instruction, Multiple Threads (SIMT) execution models of modern GPUs, they induce severe warp divergence and uncoalesced memory reads.

In this paper, I propose a paradigm shift from dynamically allocated tensor graphs to a **fixed biophysical synapse budget (Type-B Connectome)** residing on a **stationary (2D) memory substrate**. By

guaranteeing that every neuron possesses an invariant degree of B connections, warp divergence is rendered physically impossible. Additionally, a batch major row format is introduced efficiently, circumventing the memory bottleneck of the VRAM.

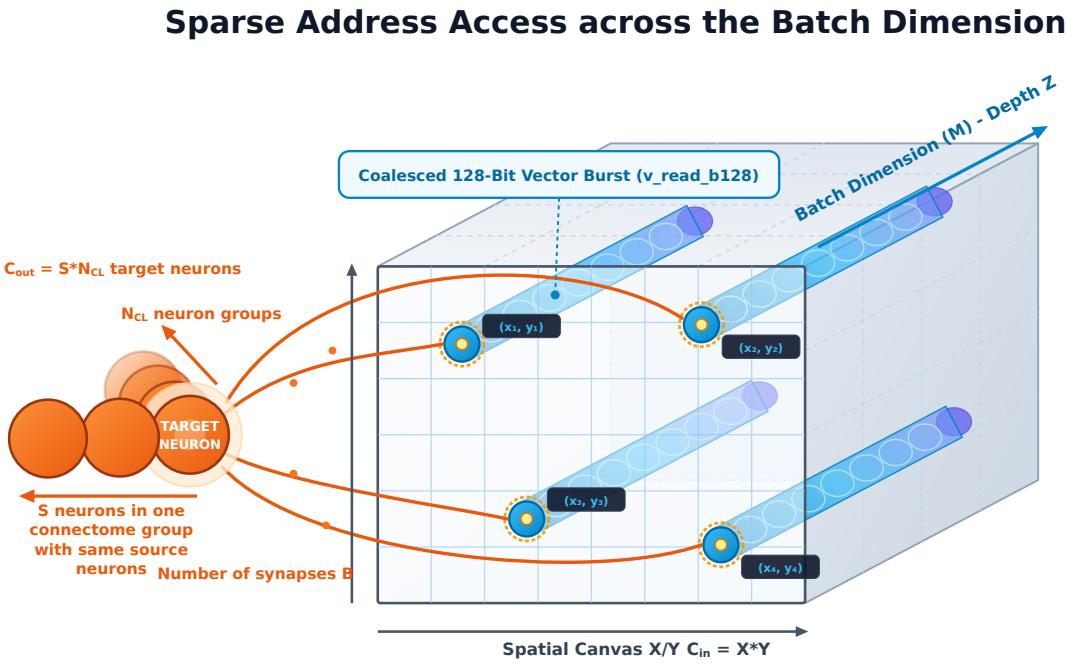


Figure 1: Sparse Address Access across the Batch Dimension. Unstructured spatial access (X/Y) is massively parallelized through coalesced 128-bit bursts along the batch axis (M). Details to the architecture are given in Section 1.3.

1.2 2. Related Work

Unstructured Sparsity and Pruning: Early work by Han et al. (2015) highlighted that deep neural networks could be pruned significantly without loss of accuracy. While algorithmic methods for finding sparse sub-networks have flourished, translating theoretical FLOP reductions into wall-clock speedups on GPUs remains challenging (Gale et al., 2019). Standard SpMV (Sparse Matrix-Vector multiplication) routines are memory-bound and struggle with irregular memory access patterns. Even dynamic sparse training topologies with fixed degree budgets such as SET (Mocanu et al., 2018) and RigL (Evci et al., 2020), or fine-grained GPU kernels like Sputnik (Gale et al., 2020), either fall back to dense masks or remain restricted to scalar SIMT pipelines without Matrix Core acceleration.

Structured and N:M Sparsity: To circumvent the hardware inefficiencies of unstructured sparsity, hardware vendors have introduced structured sparsity constraints. NVIDIA's Ampere architecture introduced 2:4 structured sparsity, which accelerates execution via Sparse Tensor Cores (Mishra et al., 2021). While highly efficient, structured sparsity heavily constrains the algorithmic freedom of the pruning process, often leading to sub-optimal model fidelity compared to unconstrained unstructured sparsity. Similarly, coarse 2D block-sparse formats (8×8 to 128×128; Gray et al., 2017; Tillet et al., 2019), structured Butterfly/Monarch factorizations (Dao et al., 2019, 2022), and token-level Mixture-of-Experts routing (MegaBlocks, Gale et al., 2023; DeepSeekMoE, Dai et al., 2024) either restrict fine-grained input selection or operate only at the macroscopic expert level.

Memory Formatting and Coalescing: Optimizing the memory layout of sparse matrices is a long-standing area of research. Formats like ELLPACK (ELL) pad rows to a fixed length to enable vectorized memory access but suffer from excessive memory overhead when row lengths are highly skewed (Bell & Garland, 2008). Our Type-B Connectome conceptually aligns with fixed-degree constraints but avoids zero-padding overhead by enforcing the budget algorithmically during the pruning or routing phase. Unlike recent Tensor Core approaches that stage raw activations into shared memory (LDS) for dense expansion (Flash-LLM, Xia et al., 2023), online micro-tile permutation (PIT, Zheng et al., 2023), or narrow 1D-vector BCRS transposition ($V \leq 8$, Magicube, Li et al., 2022), DG-WMMA transposes gathered tokens directly in VGPRs (`0 KB LDS` at $S = 16$) and uses the LDS at $S \geq 32$ solely as a conflict-free broadcast ring buffer for already-packed matrix fragments across up to 8 wavefronts ($S \leq 512$).

1.3 3. The (2D-)VRAM Connectome Architecture

Our framework is constructed upon four microarchitectural pillars that collectively bypass the Memory Wall and eliminate execution divergence. For the following explanations refer to Fig. 1 for a visual representation.

1.3.1 3.1 Pillar 1: Overcoming the SIMT Hardware Barrier

Modern GPUs execute threads in bundles known as wavefronts (AMD, 32 or 64 threads) or warps (NVIDIA, 32 threads). When standard CSR formats are evaluated, each thread corresponds to a row in the sparse matrix. Because the number of non-zeros N_{nz} per row varies wildly, threads within the same wavefront must execute heterogeneous loop iterations. The GPU hardware manages this by masking out (disabling) threads that have completed their work early, while the wavefront as a whole waits for the thread with the highest N_{nz} to finish. This leads to **warp divergence**, destroying hardware utilization.

1.3.2 3.2 Pillar 2: The Structural Guarantee (Type-B Connectome)

To physically guarantee divergence-free execution, I define the Type-B Connectome. I constrain the sparse topology such that the in-degree of every post-synaptic neuron j is exactly equal to a hyper-parameter B:

$$|\{i \mid W_{j,i} \neq 0\}| = B \quad \forall j$$

Consequently, when mapping neurons to threads within a wavefront, all 32 (or 64) SIMT lanes iterate through exactly B synapses. This enforces 100% synchronous SIMD-lockstep, completely eliminating control-flow branch penalties and pipeline bubbles.

1.3.3 3.3 Pillar 3: Memory Symmetry (2D-Interlaced Layout and Vector8 Packing)

Standard matrix representations evaluate a batch of M tokens by gathering activations across disparate memory locations, resulting in uncoalesced memory accesses. I resolve this via a **batch major orientation**.

Tokens are grouped contiguously along the channel dimension. For $M = 128$ tokens evaluated in FP16 (2 Bytes/token), the required contiguous block is exactly $128 \times 2 = 256$ Bytes. This allows for full page caching on modern GPU architectures (compare fig. 5).

Furthermore, to freely address multiple neurons, I chose to have an addressing pointer (UInt32 addressing the full Neural space) and partner (FP16). These are packed separately and may be loaded in 128 bit chunks to optimize transfer speeds.

1.3.4 3.4 Pillar 4: Small synaptic groups (SSG)

Not only the data transfer to the GPU needs to be optimized, to use the superior matrix multiplication feature of the GPU it is essential, that the neurons build small groups of S neurons reaching out for the same source neuron pattern in the connectome. These groups share one set of address pointers (UInt32), but have all their independent weights and biases. The minimal group size due to the matrix multiplication on AMDs RDNA4 WMMA is S=16.

Finally, some more parameters are of relevance:
- "C_{in}": These are the number of (potential) input Neurons available.
- "C_{out}": These are the number of output Neurons.
- "N_{CL}": Number of clusters (SSGs) such that $C_{out} = S \cdot N_{CL}$.
- "X": The input activation tensor of shape $C_{in} \times M$ containing the FP16 activity values of all input neurons across the batch of M tokens (stored in contiguous 64-token batch-major tiles).
- "Syn/s": This is the rate at which Synapses are processed. Typically, this should be roughly half the MulAdd Flops. Therefore, 1 TSyn/s corresponds to 2 TFlops.

1.3.5 3.5 On-the-fly Register Transposition & Data Reuse

Because tokens are loaded contiguously across the batch dimension (M), they arrive in the Vector General Purpose Registers (VGPRs) in a transposed orientation relative to the requirements of the Wave Matrix Multiply Accumulate (WMMA) instructions. To resolve this, our kernel performs an **on-the-fly lane permutation (Register Transposition)** directly within the VGPRs. True to this direct cache-to-register design, data transfers into and out of the kernel occur directly through the hardware cache controllers without staging raw, unprocessed global memory data in the Local Data Share (LDS). At $S = 16$, the kernel requires `0 KB LDS`; for larger groups ($S \geq 32$), the waves of a workgroup cooperatively split the sub-word token gather across two K-steps and use a small `8-16 KB` LDS ring buffer solely to broadcast already register-transposed WMMA fragments across the cluster.

Fundamentally, the VRAM bandwidth is the ultimate limiting factor of the system. Without strategic data reuse, the ~ 288 GB/s VRAM bandwidth of the RX 9060 XT is exhausted almost instantly, leaving the massive compute capabilities of the GPU severely underutilized (starving for data, compare fig. 3 for $C_{in} > 32k$ and $S = 16$) and fig. 2. Therefore, the number of VRAM queries must be cleverly reduced.

In our architecture, there are two vital “partners”: the batch tokens and the synaptic weights. By grouping

synapses into spatial clusters, multiple synaptic weights reuse the exact same batch tokens, while the batch dimension reuses the same synaptic weights across tokens—reducing the effective VRAM traffic per synapse proportionally to $\frac{1}{S} + \frac{1}{M} + \frac{1}{B}$ (token load, weight load, and output writeback). Our empirical profiling confirms that at a cluster size of $S \geq 128$, this data reuse artificially boosts the arithmetic intensity just enough to overcome the memory wall (crossing the hardware's memory-to-compute break-even point; compare fig. 5). By drastically reducing redundant VRAM fetches, the GPU compute units sustain high utilization even in out-of-cache VRAM-Streaming (fig. 3 $C_{in} > 32k$ and $S \geq 128$).

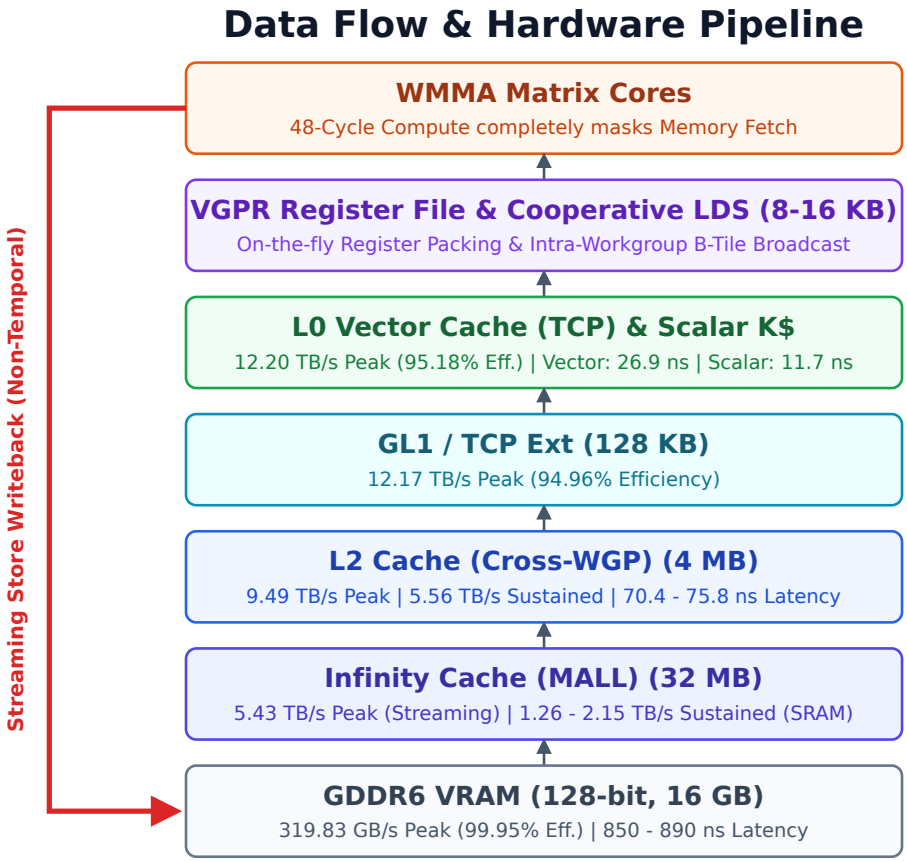


Figure 2: Data Flow & Hardware Pipeline on the AMD Radeon RX 9060 XT (16 GB). The architecture explicitly bypasses the LDS for raw memory transfers, relying on hardware controllers for transparent caching, followed by on-the-fly register transposition (and intra-workgroup LDS fragment broadcasting for $S \geq 32$). Empirical hardware metrics (latency and sustained bandwidth limits) were measured using GPUPerfTests [10].

1.4 4. Empirical Results

Instead of relying on theoretical metrics, the efficiency of the dendritic Connectome (DG-WMMA) is validated through direct silicon telemetry on the AMD Radeon RX 9060 XT.

As demonstrated in the cluster scaling benchmarks (see Figure 3), the architecture exhibits remarkable throughput scaling. Full bandwidth saturation and peak throughput are strictly attained at $S \geq 128$, confirming that the synergistic partnership between batch tokens and clustered synaptic weights is correctly managed by the hardware memory controllers. Specifically, using a 256 MiB Multi-Wave VRAM-Streaming ring buffer at the baseline configuration ($M = 512, B = 1024, C_{out} = 8192$), $S \geq 64$ reaches 29.5–30.5 TSyn/s (57%–59% of peak, 59.0–61.0 TFlops) in-cache ($C_{in} \leq 32$k), while in true VRAM-Streaming up to $C_{in} = 10$M (99.99% sparsity) $S = 128$ and $S = 256$ sustain 12.4 TSyn/s (24.7 TFlops) and 18.0 TSyn/s (35.9 TFlops), respectively.

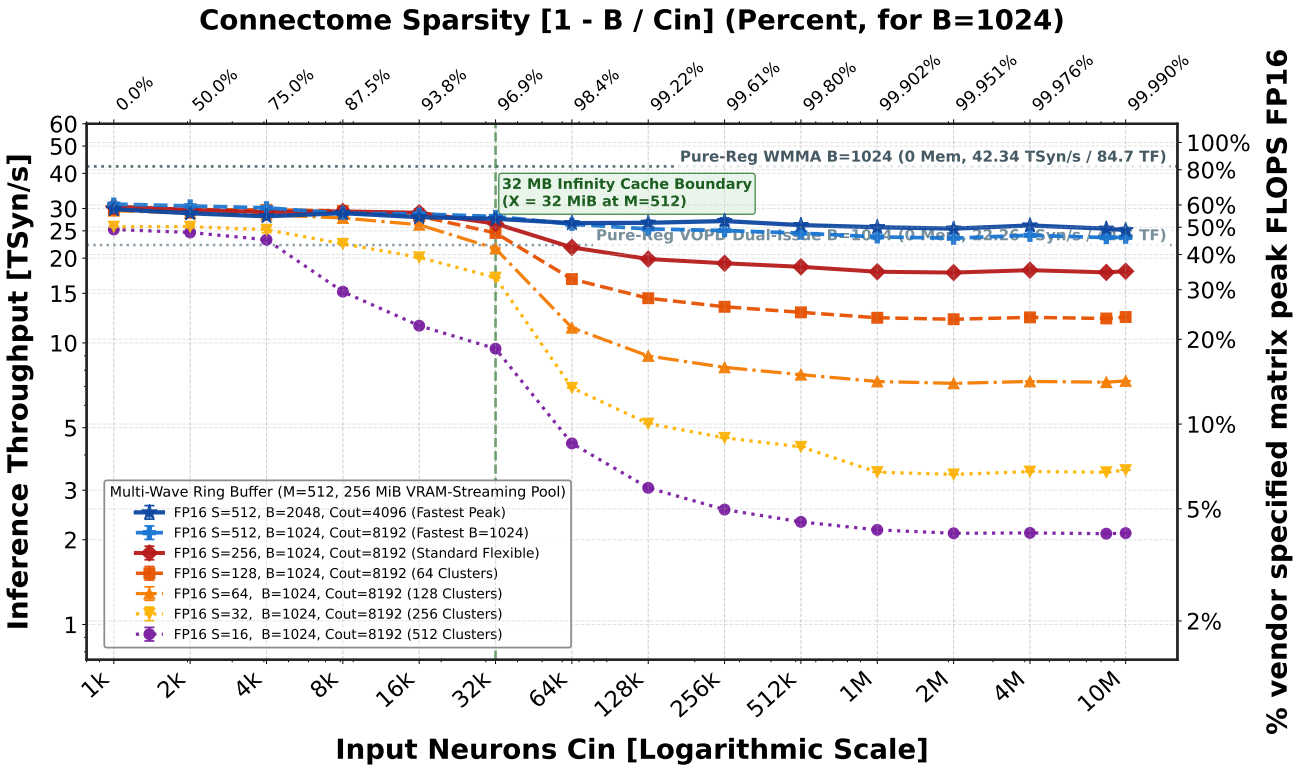


Figure 3: Throughput and Latency Scaling ($S \geq 128$). Full GPU saturation is achieved when sufficient data reuse occurs between batch tokens and synaptic weights. The x-axis represents the number of input connections (c_{in}), while the y-axis shows the throughput in TSyn/s (left) relative to theoretical max speed (right) on a logarithmic scale. The convergence of the throughput curves at larger cluster sizes demonstrates that the VRAM bottleneck is successfully mitigated through token reuse. Calculation speed of (self implemented) pure-register benchmark kernels issuing the VALU speed and the WMMA speed without memory overhead etc. are shown as horizontal dotted lines and may guide the expectations of what an ideal DNN could reach on this hardware.

Reducing the group size of the neurons to S=64, 32 and 16 maintains high throughput for smaller C_{in} sizes up to approx. 94%–97% sparsity (while the input activation tensor X fits in the 32 MB Infinity Cache). But for larger C_{in} limitations in VRAM bandwidth kick in (compare fig. 2). This is expected since smaller group sizes limit the number of reusable token values in the GPU cache. So if the data is too large for the cache performance drops significantly.

For full speed evaluation two $S = 512$ ($M = 512$ batch size) comparison curves ($B = 1024, C_{out} = 8192$ and $B = 2048, C_{out} = 4096$) were plotted as well. Here up to 60% of theoretical Syn/s rates (31.1 TSyn/s / 62.2 TFlops) are achieved in-cache for sparsity from 0 to approx. 94%. Nevertheless, high speeds around 23.7–25.2 TSyn/s (or 46%–49% of theoretical max. of the hardware, > 50 TFlops) are sustained in VRAM-Streaming up to $C_{in} = 10$M (99.98%–99.99% sparsity) until the VRAM size limits further scaling. For $M = 512$ this happens when $C_{in} > 10$M (corresponding to activation mem usage > 10 GB alongside the ring buffer).

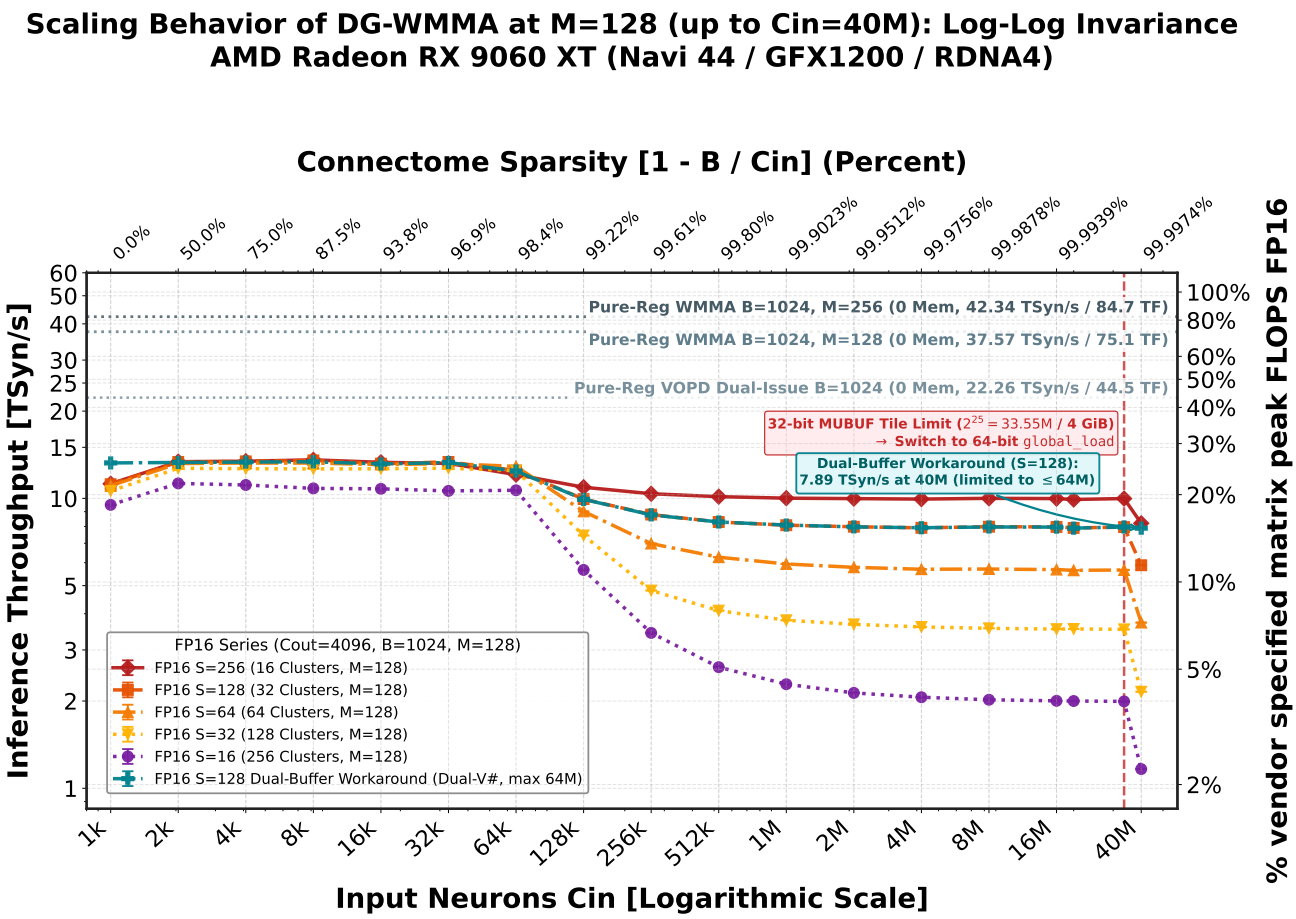


Figure 4: Extended Cluster Scaling at $M = 128$ up to $C_{in} = 40$M (99.9974% sparsity). Reducing the batch depth to $M = 128$ increases the neuron capacity of the 32 MB Infinity Cache and allows scaling past the 32-bit MUBUF descriptor limit ($2^{25} = 33.55$M neurons / 4 GiB per 64-token tile, vertical red dashed line), where DG-WMMA switches from 32-bit `buffer_load` to 64-bit `global_load` addressing. Also shown is a demonstration curve for $S = 128$ using the Dual-Buffer (`Dual-V#`) hardware workaround, which bridges the 4 GiB boundary up to 64M neurons.

To evaluate scaling up to $C_{in} = 40$M (99.9974% sparsity) within the 16 GB VRAM limit, Figure 4 repeats the cluster sweep at $M = 128$ ($C_{out} = 4096, B = 1024$). Reducing the batch size to $M = 128$ lowers the memory footprint per neuron (256 Bytes), increasing the number of neurons that fit into the 32 MB Infinity Cache up to $C_{in} = 64$k–128k (reaching 13.5–13.6 TSyn/s in-cache for $S = 128$–256). Furthermore, scaling past $C_{in} = 2^{25} \approx 33.55$M neurons exposes the 4 GiB addressing limit of RDNA4's 32-bit `buffer_load` (MUBUF) hardware descriptors (at 128 Bytes per 64-token slice). Beyond 33.55M, DG-WMMA automatically switches to 64-bit `global_load` addressing, sustaining 5.9 TSyn/s ($S = 128$) and 8.2 TSyn/s (16.4 TFlops at $S = 256$) at 40M neurons. While a Dual-Buffer descriptor workaround (`Dual-V#`, cyan curve, 7.9 TSyn/s at 40M) can avoid this step-down across the 4 GiB boundary, it doubles the MUBUF instruction rate and remains capped at $2^{26} \approx 67.1$M neurons.

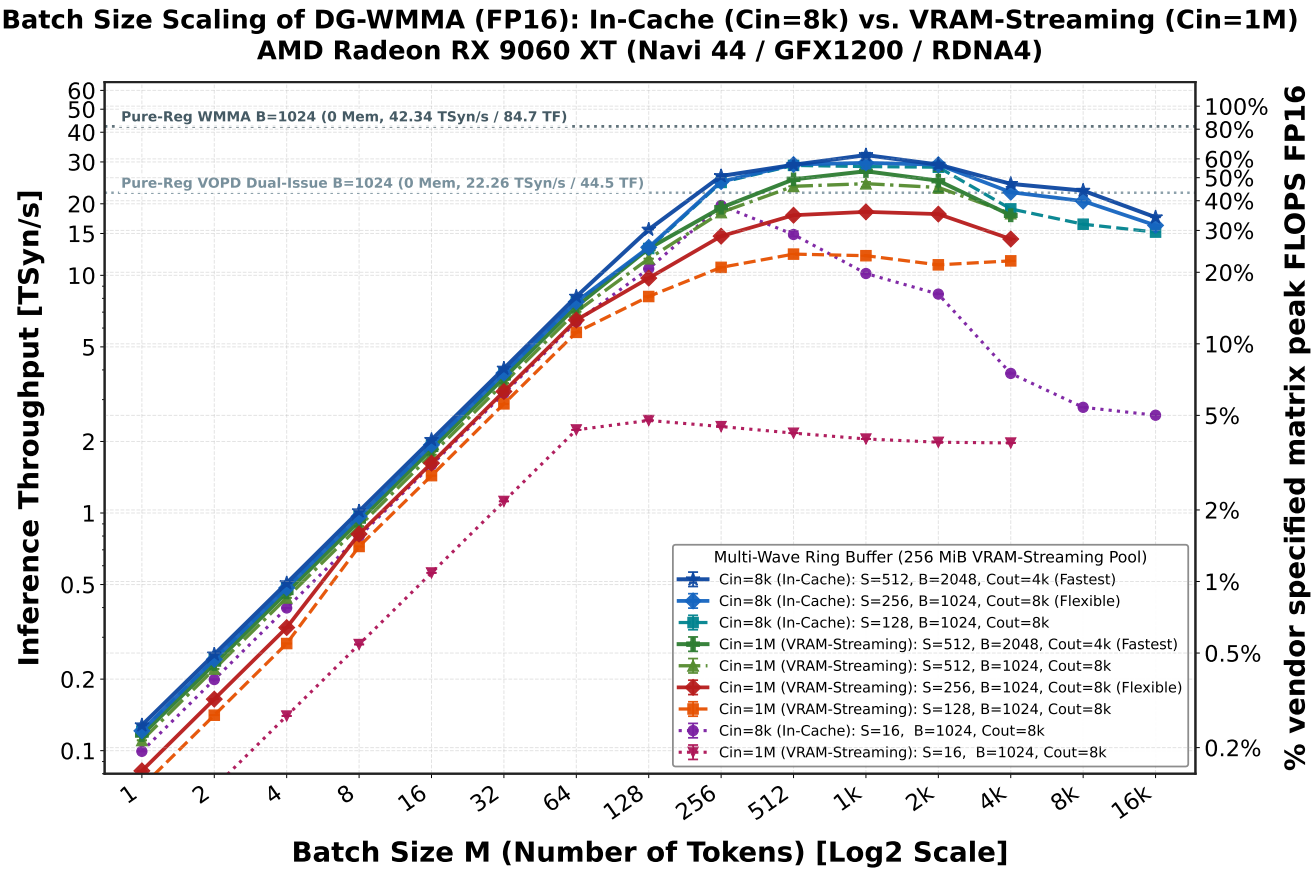


Figure 5: Batch Scaling Benchmark. The logarithmic increase in throughput underscores the importance of the 3D-Batch architecture for unstructured memory access. The horizontal axis indicates the batch depth (M), while the vertical axis shows the achieved throughput (TSyn/s). As batch size increases, the memory access latency per token is perfectly amortized via 128-bit vector bursts, allowing the system to scale efficiently until hardware computational bounds are reached. Again the horizontal dotted lines guide the expectations of what an ideal DNN could reach on this hardware. (compare fig. 3)

Furthermore, the performance exhibits heavy dependence on the batch dimension, which enables the coalesced `v_read_b128` bursts. This logarithmic scaling of throughput relative to the batch depth is independently confirmed in Figure 5. Remarkably, the turnover point from batch size (so VRAM load limited) to calculation limited is at $M = 128$. This corresponds to 256 byte chunks and is obviously larger than the actual 128-bit memory calls. I assume this to be caused by the page size of the VRAM or a similar limitation, which makes the VRAM not truly random access. Nevertheless, given the correct batch size this limitation can be circumvented. Comparing In-Cache ($C_{in} = 8$k) against VRAM-Streaming ($C_{in} = 1$M) in Figure 5 shows that both regimes reach their optimal plateau at $M = 256$–1024 (up to 32.1 TSyn/s / 64.1 TFlops in-cache

and 18.5–27.4 TSyn/s / 37.0–54.8 TFlops for $S = 256$–512 in VRAM-Streaming), before the active token footprint at $M \geq 4096$ exceeds the 32 MB Infinity Cache.

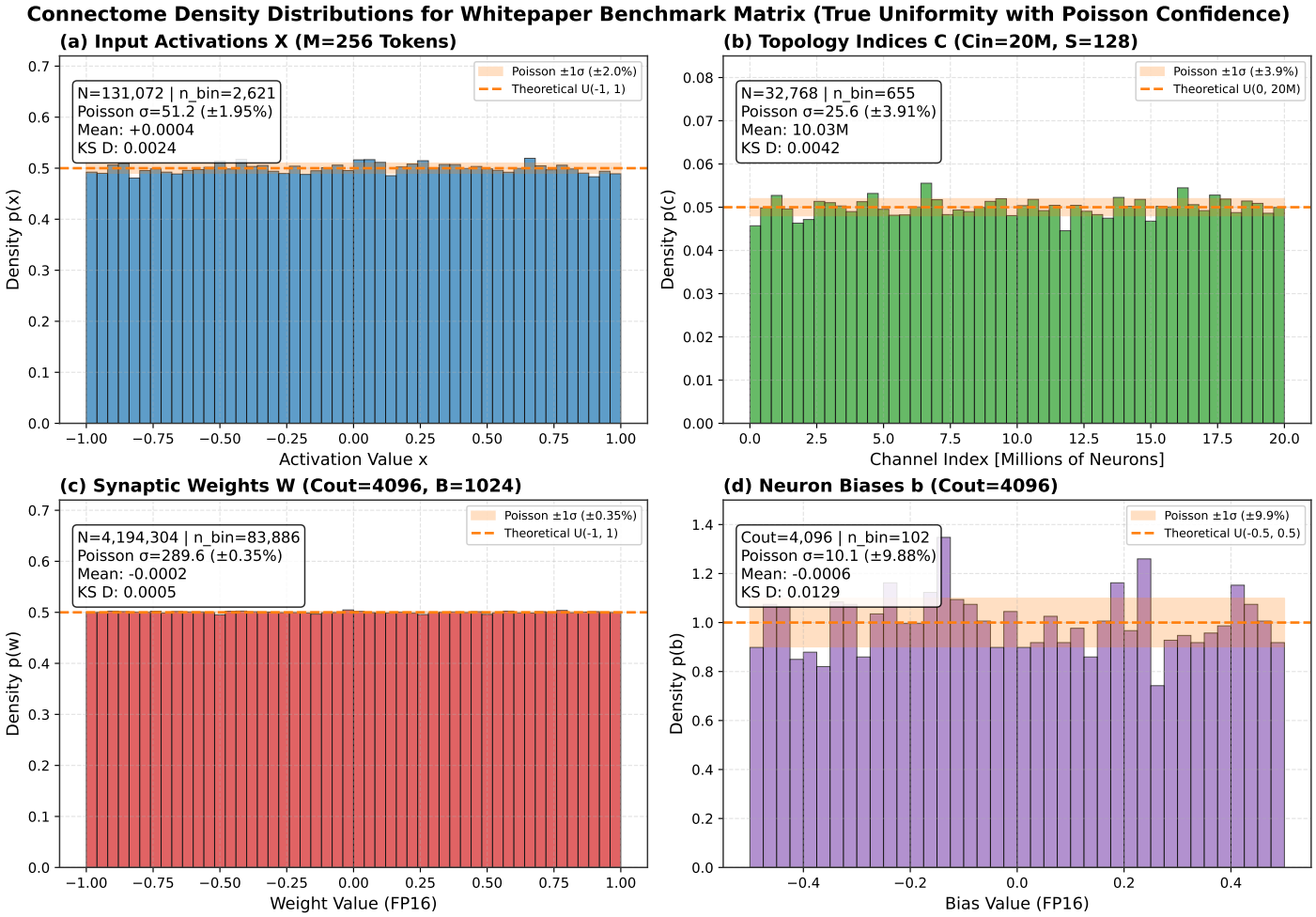


Figure 6: Overview of the distributions to verify the random nature of the parameters used for the benchmark. Expected mean value and standard deviation $\sqrt{n}/n$ for average bin numbers (n) are indicated in all plots. KS D is Kolmogorow-Smirnow-Distance (a) Distribution of the activity values of the C_{in} neurons. (b) Address-distribution from the selected input neurons. (c) Distribution of the weights over all synapses. (d) Biases of all output Neurons.

Since I do not really trust the AI agents, I let them implement a statistical analysis of the "randomly" generated values for input neuron activity, address space accessed by the synapses, weight distribution and bias distribution (compare Fig. 6). Those values were used to compare to a CPU implementation of the sparse calculations to verify the identical outcome and omit shortcuts. The range of values (mean and std.) are indicated to verify the distribution by eye. The Kolmogorow-Smirnow-Distance (KS D) gives us a number and

as far as I can tell the number appears to confirm a uniform distribution with Poisson-distributed bin counts. All tests and graphs in this manuscript may be reproduced by using the provided code in the repository.

1.5 5. Discussion

1.5.1 5.1 Sparsity and size of addressable Neurons

A common misconception in unstructured sparse architectures is that the performance or memory footprint scales directly with the raw number of non-zero synapses. However, in our (2D)-VRAM Connectome, the sparsity-induced overhead scales primarily with the size of the addressable neuron space (the neural canvas). Because the Type-B connectome mandates a fixed synapse budget (B) per neuron, the memory footprint and the bandwidth requirements are strictly deterministic. The true sparsity ratio is governed by how large the target neuron population is relative to B. This means that scaling to massive network sizes requires only proportional scaling of the addressable canvas, completely avoiding the exponential explosion in synaptic index tracking that plagues traditional sparse formats like CSR.

Furthermore, this opens up a completely new approach for NN-Architectures. Instead of thinking in layers defining some group Neurons for C_{in} and another for C_{out} and the layering of those. I may use optimization algorithms which will "decide" on their own, which neurons to choose for the "previous" step. The neuron groups will fire in waves each choosing their own, relevant connections by this forming an optimized architecture. And alternating e.g. using back propagation optimizing those connections weights and biases. Because neurons are already bundled into clusters of size S, we can select the ideal hardware parameters (S, M, B, and C_{out}) for each individual kernel call and represent nearly any total C_{out} size (or larger synapse budgets B) simply by chaining multiple calls writing to distinct target offsets in memory. The only requirement is that neurons within a single call do not depend on each other's current-step results; since DG-WMMA accumulates in registers and writes to global memory only at the very end of the kernel, previous-step values can even be read directly across the full C_{in} space. Of course it can be used to sparsify existing NNs and that will be the first logical step. But there is no technical reason to largely increase C_{in} such that every neuron group (similar to Minicolumns in biology) can reach out to the most relevant information available in the whole connectome—with 40 million addressable neurons on a single GPU already approaching the complexity scale of a mouse brain (*Mus musculus*, ~ 70 million neurons).

1.5.2 5.2 The Systems and Hardware-Co-Design Enabler Paradigm

A frequent critique from the machine learning community regarding extreme unstructured sparsity (e.g., 90%) is that it may severely degrade the perplexity and downstream task performance of LLMs if applied naively. I explicitly state that the (2D)-VRAM Connectome framework does not profess to have unilaterally solved the algorithmic accuracy degradation inherent to 90% sparse networks.

Instead, this paper positions the framework as the **missing high-performance Systems Foundation**. Historically, the "chicken-and-egg" problem of sparsity dictated that researchers had no economic incentive to engineer algorithms capable of preserving perplexity at 90% sparsity, because standard hardware could not accelerate them efficiently. By definitively proving that $> 90\%$ unstructured sparsity can yield effective hardware usage, speedups of sparse systems over 10x—surpassing even dedicated dense matrix multiplication hardware—we provide the ML community with a new tool. Researchers now have the opportunity and the tools to develop ultra-sparse algorithms (up to 99.997%), knowing they can be profitably deployed. If large batch sizes can be introduced for a problem even in production edge environments.

1.5.3 5.3 Algorithmic Flexibility vs. Structured Constraints

Traditional frameworks (e.g., PyTorch) and structured sparsity constraints (e.g., N:M patterns) restrict neural network topologies to conform to rigid dense block structures. The (2D)-VRAM approach diverges from this trend. By coupling unstructured sparsity directly with the physical realities of the GPU architecture (cache topologies, wavefront alignment, and VGPR limits), the fixed B-budget paradigm grants researchers vast algorithmic freedom regarding *where* synapses are placed, demanding only that the *quantity* of synapses remains deterministic. This determinism is sufficient to recover the computational efficiency traditionally reserved for dense execution.

1.5.4 5.4 A Taxonomy of Sparse SIMT Parallelization

Rather than prescribing a single universal architecture, our findings suggest a taxonomy of sparse SIMT parallelization strategies tailored to specific operational regimes:

Minicolumn Sparsity (Spatial Wavefront-Alignment): By clustering $S = 16$ distinct neurons into a single wavefront (or $S \in \{32, 64, 128, 256, 512\}$ across a workgroup via the LDS broadcast ring buffer) and enforcing shared connectivity, I replicate the biological concept of cortical mini-columns. This enables the use of dense Matrix-Cores (WMMA). In our empirical tests, Minicolumn Sparsity achieved a staggering **25 to 32.1 TSyn/s physical throughput** in-cache on RDNA4 (and **18.0 to 27.4 TSyn/s** in VRAM-Streaming for $S = 256$–512), corresponding to (e.g. at 90% sparsity) an effective dense throughput of over **250–320 TSyn/s**.

While Minicolumn Sparsity currently reaches up to 32.1 TSyn/s (approximately 62% of the theoretical 51.5 TSyn/s WMMA hardware limit), the primary bottleneck is no longer the arithmetic intensity, but the structural alignment of the gathered tokens. After the scattered tokens are gathered from VRAM, they reside in a transposed layout in the token stacks.

1.5.5 5.5 Hardware Utilization vs. Algorithmic Efficiency

In high-performance computing, unstructured sparse algorithms typically suffer from catastrophic hardware underutilization, often falling below 5% of the silicon's theoretical peak floating-point performance due to memory stalls. A heavily optimized dense matrix multiplication (GEMM) can typically achieve ~80% hardware utilization (matching our pure-register zero-memory-transfer WMMA microbenchmarks at $B = 1024$ in Figs. 3–5). Remarkably, our $S = 16$–512 Minicolumn Sparse Connectome reaches a physical throughput of 25 to 32.1 TSyn/s in-cache (roughly 49%–62% of the RX 9060 XT's absolute theoretical WMMA ceiling of 51.5 TSyn/s) and sustains 18.0 to 27.4 TSyn/s (36.0–54.8 TFlops for $S = 256$–512) in out-of-cache VRAM-Streaming. Achieving over half the raw hardware efficiency of a perfect dense kernel while operating on 90% to 99.98% sparsity (and scaling up to 99.997% sparsity at 40M neurons), structurally irregular data is unprecedented. Furthermore, because 90% of the mathematical operations are circumvented algorithmically, this 49%–62% physical utilization yields a >6x wall-clock speedup over the absolute theoretical maximum of a dense kernel with zero weights.

1.5.6 5.6 Outlook: Universal Execution of Real-World Networks

As a next step toward the general-purpose **SNNtome** framework, an extended token-shift variant of the kernel and an automated graph-to-wave translator (`WaveCompiler`) were developed to evaluate whether the stationary 2D-canvas connectome can execute complete, unmodified real-world dense networks (2D CNNs,

U-Nets, YOLOv8x object detectors, and autoregressive LLMs) inside a single pre-allocated `d_Canvas` buffer with zero external workspace. Five targeted compile-time generalizations were required:

1. **2-Byte Token-Shift Gather Addressing (`v_lshl_add_u32`):** Replacing the 128-Byte channel stride shift (`« 7`) with a single-`FP16` offset (`(cols[s] « 1) + m_off`) allows a single precomputed index table to unify 1×1 and 3×3 spatial convolutions ($\Delta x, \Delta y$), zero-copy feature concatenations (`Concat`), and 1D linear projections in one instruction.
2. **Cross-Row Tile-Wrapping ($W_{\mathbf{pad}} = W + 4$) & `PER_TILE_MASK`:** Wrapping 64-token tiles across row boundaries with a 4-pixel zero guard band and a branchless 16-bit lane validity mask eliminates up to 60% of wasted tiles on small feature maps (e.g., 20×20) without `hipMemset` calls between frames.
3. **In-Register Epilogue Fusion & Unbiased `RTE` Rounding:** Bias, activation (`ReLU`, `Sigmoid`, `SiLU`, `LeakyReLU`), and skip-residual additions are fused directly in `FP32` registers prior to unbiased IEEE-754 Round-To-Nearest-Even (`RTE`) downcast.
4. **Multi-Wave In-Place Self-Read Accumulation (I_S):** Layers whose fan-in exceeds a single wave's budget ($B > B_{\max}$) are split across sequential waves without atomics or workspace buffers by reserving the first S synapse slots for an in-place self-read weighted by the $S \times S$ identity matrix I_S.
5. **Polyphase Stride-2 (`S2D`) & Compile-Time `RMSNorm` γ-Folding:** Stride-2 downsampling convolutions are reorganized into $4 \cdot C_{\text{in}}$ polyphase channels, while Transformer `RMSNorm` scale vectors γ are folded into the subsequent wave's weights at compile time.

Table 1 summarizes the end-to-end verification results (at 100% density, prior to any pruning) against industry-standard engines (`ROCm MIOpen 3.5 FP16` with `exhaustiveSearch = true`, `NCNN Vulkan FP16`, and `Ollama / llama.cpp F16`) on the same AMD Radeon RX 9060 XT.

Table 1: End-to-end real-world verification of `Kernel 44` (`0 B` workspace, 100% dense baseline) vs. industry SOTA engines on the AMD Radeon RX 9060 XT.

Model / Workload	Kernel 44 (FP16)	Competitor Baseline (Same GPU)	Speedup	Fidelity vs. Ref.
Brain MRI U-Net	**0.855 ms** (1,170 FPS,	MIOpen FP16 Ex.: 1.338 ms (747 FPS)	**1.56**×	100.00% Dice / IoU
(23 Layers, 256×256×3)	28.3 TF, 23 Waves)	NCNN Vulkan FP16: 2.134 ms (469 FPS)	**2.50**×	99.9999% CosSim
YOLOv8x Detection	**7.259 ms** (137.8 FPS,	MIOpen FP16 Ex.: 11.220 ms (89.1 FPS)	**1.55**×	100.0000% CosSim
(104 Layers, 640×640×3,	35.5 TF; Video: **8.08 ms**	MIOpen Video: 14.011 ms (71.4 FPS)	**1.73**×	99.98% Box IoU
Static & 46,530-Frame Video)	/ 123.8 FPS)	NCNN Vulkan FP16: 14.895 ms (67.1 FPS)	**2.05**×	100% Class Match
SmolLM-135M	Decode: **3.34 ms** (300 Tok/s)	MIOpen FP16 Ex.: 9.94 ms (101 Tok/s)	**2.98**×	99.9998% CosSim
(30 Layers, 134.5M Params)	Prefill T=512: **4.51 ms** LinW	Ollama F16: 3.3 ms Dec. / 15.89 ms Pref.	**3.52**× (LinW)	100% Token Match
Llama-3.2-3B-Instruct	Decode: **25.85 ms** (38.7 Tok/s,	Ollama F16: 23.2–27.1 ms Decode	Parity (Dec.)	99.9999% CosSim
(28 Layers, 3.21B Params,	268.8 GB/s = 93.3% BW Peak)	Ollama F16 Prefill T=64: 43.92 ms	**1.47**× (Pref.)	PPL: 53.21 vs. 53.20
6.43 GB FP16 Streaming)	Prefill T=512: **141.0 ms** (44.9 TF)	Ollama F16 Prefill T=512: 180.42 ms	**1.28**× (3.82× LinW)	100% Token Match

Finally, to verify that the DG-WMMA connectome paradigm is vendor-agnostic, a native CUDA port is currently in preparation for future release in the **SNNtome** repository. Preliminary benchmarks confirm that the same cluster-gather and Matrix-Core design achieves up to 111.8 TSyn/s (223.5 TFlops, 61.4% of theoretical dense Tensor Core peak) in-cache and 78.4 TSyn/s (156.9 TFlops) in VRAM-Streaming at $C_{\text{in}} = 10$M on an NVIDIA RTX 6000 Ada (`sm_89`, scaling to 40M neurons at 30.3 TSyn/s), as well as up to 612.7 TSyn/s (1.23 PFlops in-cache; 522.4 TSyn/s / 1.04 PFlops in HBM3e-Streaming at $C_{\text{in}} = 80$M, and scaling up to 320M neurons) on an NVIDIA Blackwell B200 (`sm_100a`) using 5th-generation `tcgen05.mma` instructions and Tensor Memory (`TMEM`)—reaching up to 981.2 TSyn/s (1.96 PFlops) when orthogonally combined with hardware 2:4 weight sparsity. Because the CUDA port is still in preparation, these figures should be regarded as preliminary.

1.6 6. Conclusion

The transition from dynamically allocated tensor graphs to biologically inspired, stationary hardware connectomes represents a fundamental paradigm shift in AI execution. By enforcing a fixed biophysical synapse budget and optimizing 128-bit memory strides through a 2D-interlaced layout, the (2D)-VRAM Connectome completely eliminates warp divergence and largely bypasses the Memory Wall. Reaching up to 62% of the theoretical Matrix Core peak in-cache and sustaining over 50 TFlops out-of-cache (at $S = 512$), this architecture firmly establishes itself as a premier systems infrastructure, paving the way for the next generation of highly sparse neural networks.

1.7 References

1. Bell, N., & Garland, M. (2008). Efficient sparse matrix-vector multiplication on CUDA. *NVIDIA Technical Report NVR-2008-004*.
2. Frankle, J., & Carbin, M. (2018). The lottery ticket hypothesis: Finding sparse, trainable neural networks. *ICLR 2019*.
3. Gale, T., Elsen, E., & Hooker, S. (2019). The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*.
4. Han, S., Pool, J., Tran, J., & Dally, W. (2015). Learning both weights and connections for efficient neural network. *NeurIPS 2015*.
5. Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. *NeurIPS 2022*.
6. Jouppi, N. P., Young, C., Patil, N., et al. (2017). In-datacenter performance analysis of a tensor processing unit. *ISCA 2017*.
7. Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
8. Mishra, A., Latorre, J. A., Pool, J., et al. (2021). Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*.
9. Patterson, D., Anderson, T., Cardwell, N., et al. (1997). A case for intelligent RAM. *IEEE Micro, 17*(2), 34-44.
10. Nemez, Chips and Cheese (2024). GPUPerfTests. *GitHub repository*. https://github.com/ChipsandCheese/GPUPerfTests
11. Mocanu, D. C., Mocanu, E., Stone, P., Nguyen, P. H., Gibescu, M., & Liotta, A. (2018). Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications, 9*, 2383.
12. Evci, U., Gale, T., Menick, J., Castro, P. S., & Elsen, E. (2020). Rigging the lottery: Making all tickets winners. *ICML 2020* (arXiv:1911.11134).
13. Gale, T., Zaharia, M., Young, C., & Elsen, E. (2020). Sparse GPU kernels for deep learning. *SC20*.
14. Gray, S., Radford, A., & Kingma, D. P. (2017). GPU kernels for block-sparse weights. *OpenAI Technical Report* (arXiv:1711.09224).
15. Tillet, P., Kung, H. T., & Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. *MAPL 2019*.
16. Dao, T., Gu, A., Eichhorn, M., Rudra, A., & Ré, C. (2019). Learning fast algorithms for linear transforms using butterfly factorizations. *ICML 2019*.
17. Dao, T., Chen, B., Sohoni, N., et al. (2022). Monarch: Expressive structured matrices for efficient and

accurate training. *ICML 2022* (arXiv:2204.00595).
18. Gale, T., Narayanan, D., Young, C., & Zaharia, M. (2023). MegaBlocks: Efficient sparse training with mixture-of-experts. *MLSys 2023* (arXiv:2211.15841).
19. Dai, D., Deng, C., Zhao, C., et al. (2024). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*.
20. Xia, H., Zheng, Z., Li, Y., et al. (2023). Flash-LLM: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. *PVLDB, 17*(2), 211–224.
21. Zheng, N., Jiang, H., Zhang, Q., et al. (2023). PIT: Optimization of dynamic sparse deep learning models via permutation invariant transformation. *SOSP 2023*.
22. Li, S., Osawa, K., & Hoefler, T. (2022). Efficient quantized sparse matrix operations on Tensor Cores. *SC22* (arXiv:2209.06979).